
Argo Rollouts

Déploiement progressif & rollback automatique sur Kubernetes

Plan

- 1. La technologie — problème, définition, fonctionnement**
- 2. Cas d'usage professionnel — où, comment, avantages/limites**
- 3. Mise en œuvre technique — architecture, manifests, démo live**
- 4. Retour d'expérience — bilan, avis, améliorations**

Démo 100% fonctionnelle : ArgoCD (GitOps) + Argo Rollouts + analyse automatique sur un cluster local kind.

Le problème : le Deployment classique

Un Deployment fait un rolling update : il remplace les pods un à un... sans jamais vérifier si la nouvelle version fonctionne correctement.

Une v2 buggée part vers 100% des utilisateurs
Détection tardive (alertes, support, utilisateurs mécontents)
Rollback manuel, sous stress, en plein incident

On déploie « en aveugle ». Besoin : déployer prudemment et MESURER avant de généraliser.

Qu'est-ce qu'Argo Rollouts ?

Un contrôleur Kubernetes (projet CNCF, famille Argo) qui remplace le Deployment par une ressource Rollout offrant la progressive delivery :

Canary : un % croissant du trafic vers la nouvelle version

Blue-Green : deux environnements, bascule instantanée

Analyse automatique : promotion ou rollback piloté par des métriques

Objectif : une mise en production graduelle, mesurée et réversible automatiquement.

Fonctionnement global

Rollout (remplace Deployment)

- └─ ReplicaSet STABLE (version actuelle)
- └─ ReplicaSet CANARY (nouvelle version)
- └─ AnalysisRun (mesure la santé du canary → décide)

À chaque palier (setWeight 20 → 40 → ...), le contrôleur :

ajuste la proportion de pods/trafic canary,
marque une pause (manuelle ou minutée),
lance une analyse ; si elle échoue → rollback automatique.

Les briques : Rollout + AnalysisTemplate

Rollout — la stratégie :

```
strategy:
  canary:
    steps:
      - setWeight: 20
      - pause: { duration: 30s }
      - setWeight: 40
      - analysis: { templates: [{ templateName: success-rate }] }
```

AnalysisTemplate — le critère de décision (métrique Prometheus, requête HTTP, ou Job de sonde). Hors limites → métrique Failed → Rollout avorté → retour stable.

Cas d'usage en entreprise & écosystème

Réduction du risque de mise en prod : tester la v2 sur 5-10% du trafic réel

Déploiements fréquents (plusieurs/jour) sans fenêtre de maintenance

SLO-driven delivery : promotion conditionnée à des métriques (latence, taux d'erreur, succès) issues de Prometheus, Datadog, New Relic, CloudWatch...

Couplé au GitOps (ArgoCD/Flux) : déploiement déclaratif, auditable, reproductible

Projet CNCF (Argoproj) — combo standard avec ArgoCD ; voisins : Flagger, Spinnaker

Contextes : microservices, APIs critiques, e-commerce, SaaS à fort trafic.

Avantages / Limites

Avantages

- Rollback automatique en cas de régression → moins d'incidents
- Pas de downtime, exposition contrôlée
- Décision objective (métriques) plutôt qu'« au feeling »
- Intégration native : ArgoCD, Prometheus, Istio, NGINX

Limites

- Complexité accrue (CRD, analyse, parfois service mesh)
- Contrôle fin du trafic = Istio/NGINX/SMI (sinon split par nb de pods)
- Nécessite de bonnes métriques : analyse mal calibrée = fausse confiance

Architecture de la démo

Cluster local kind + ArgoCD (déjà en place) + Argo Rollouts v1.7.2

git push → GitHub Actions (lint/validation des manifests)

└─> ArgoCD (pull) → Rollout → Contrôleur Argo Rollouts

└─ RS stable / RS canary

└─ AnalysisRun (Job de sonde HTTP)

App de démo : argoproj/rollouts-demo — une couleur par version.

Le tag spécial bad-red renvoie des HTTP 500 (version buggée déterministe).

Choix techniques clés

Stratégie canary sans service mesh → split par nombre de pods (simple, portable)

Services stable + canary : le contrôleur y injecte le pod-template-hash

→ la sonde cible précisément les pods canary

Probes TCP (et non HTTP /color) : les pods bad-red restent Ready malgré les 500

→ on démontre un rollback piloté par l'ANALYSE, pas par la santé

AnalysisTemplate type Job : 30 requêtes, exige $\geq 95\%$ de succès

(autonome ; variante Prometheus fournie pour la production)

Démo 1 — Canary SAIN (promotion auto)

**nouvelle version → canary 20%→40% ; l'AnalysisRun sonde le canary :
"30/30 requetes en succes -> 100% (seuil 95%)" → promotion automatique 100%**

```
kubectl argo rollouts get rollout - canary en cours
```

```
Name:                rollouts-demo
Namespace:           demo
Status:              [] Paused
Message:             CanaryPauseStep
Strategy:            Canary
  Step:              3/9
  SetWeight:         40
  ActualWeight:      40
Images:              argoproj/rollouts-demo:blue (canary)
                   argoproj/rollouts-demo:green (stable)

Replicas:
  Desired:           5
  Current:           5
  Updated:           2
  Ready:             5
  Available:         5
```

NAME	KIND	STATUS	AGE	INFO
[] rollouts-demo	Rollout	[] Paused	24m	
# revision:5				
[] rollouts-demo-774b46cfdc	ReplicaSet	✓ Healthy	24m	canary
[] rollouts-demo-774b46cfdc-6z47x	Pod	✓ Running	50s	ready:1/1
[] rollouts-demo-774b46cfdc-x99l2	Pod	✓ Running	14s	ready:1/1
α rollouts-demo-774b46cfdc-5	AnalysisRun	✓ Successful	14s	✓ 1
6a10cc86-ffbc-4d2d-a71c-df3687acfea2.http-success-rate.1	Job	✓ Successful	14s	
# revision:4				
[] rollouts-demo-754fbfc964	ReplicaSet	✓ Healthy	21m	stable
[] rollouts-demo-754fbfc964-tjwb5	Pod	✓ Running	21m	ready:1/1
[] rollouts-demo-754fbfc964-gh4z9	Pod	✓ Running	21m	ready:1/1
[] rollouts-demo-754fbfc964-sbjfn	Pod	✓ Running	20m	ready:1/1

Démo 2 — Canary DÉFAILLANT (rollback auto)

bad-red → HTTP 500 ; sonde 5/30 = 16% → ANALYSE KO
Metric Failed → RolloutAborted → trafic gardé sur green, 0 downtime

```
kubectl argo rollouts get rollout - rollback automatique
```

```
Name:          rollouts-demo
Namespace:     demo
Status:        * Degraded
Message:       RolloutAborted: Rollout aborted update to revision 3: Metric "http-success-rate" assessed Failed
Strategy:      Canary
  Step:        0/9
  SetWeight:   0
  ActualWeight: 25
Images:        argoproj/rollouts-demo:bad-red (canary)
               argoproj/rollouts-demo:green (stable)

Replicas:
  Desired:     5
  Current:     6
  Updated:     1
  Ready:       4
  Available:   4
```

NAME	KIND	STATUS	AGE	INFO
rollouts-demo	Rollout	* Degraded	6m21s	
# revision:3				
rollouts-demo-7c84cfdcdb	ReplicaSet	✓ Healthy	58s	canary
rollouts-demo-7c84cfdcdb-qzt5x	Pod	✓ Running	58s	ready:1/
rollouts-demo-7c84cfdcdb-hdhqt	Pod	⦿ Terminating	17s	ready:1/
rollouts-demo-7c84cfdcdb-3	AnalysisRun	* Failed	17s	* 1
021d463e-6109-4b46-93de-600d302f6668.http-success-rate.1	Job	* Failed	17s	
# revision:2				

Difficultés rencontrées & solutions

App sans /metrics → analyse Prometheus impossible

AnalysisTemplate type Job (sonde HTTP autonome)

Probe HTTP tuait les pods bad-red (500) avant l'analyse

Bascule en probe TCP : pods Ready mais erreurs mesurées par l'analyse

Cibler uniquement le canary pour la sonde

Services stable/canary gérés par le contrôleur (pod-template-hash)

Disque limité (94%)

Pas de stack Prometheus lourd : dashboard Rollouts + kind load des images

Retour d'expérience : bilan, avis, améliorations

Appris : progressive delivery en pratique (étapes, pauses, analyse, rollback) ;
imbrication GitOps ↔ Rollouts ; les métriques comme contrat de qualité.

Points forts : rollback auto fiable, intégration native, déclaratif

Points faibles : complexité, trafic fin nécessitant un mesh, analyse à calibrer

Mon avis : excellent compromis sécurité/vélocité pour des services critiques ;

overkill pour un petit projet mono-service à faible trafic.

Améliorations possibles :

Trafic fin via Istio/NGINX • analyse Prometheus réelle (p95 + erreurs) •
notifications Slack • Blue-Green avec tests preview • multi-cluster ApplicationSet

Merci — Questions ?

kubectrl argo rollouts get rollout rollouts-demo --watch

Dépôt : manifests • GitOps • CI • diagrammes • captures réelles d'exécution